

1. Il Game of Life di John Conway

Il Game of Life (GoL) si basa sull'evoluzione di un automa cellulare a dimensione zero, ovvero senza l'influenza di input esterni, che simula una popolazione di cellule su una griglia di gioco, rappresentata come una matrice bidimensionale ($N \times N$). Ogni cella ha un valore associato al suo stato: "Vivo (1)" o "Morto (0)".

Il calcolo della nuova generazione avviene considerando, per ogni cella, il suo stato attuale e quello delle otto celle a lei adiacenti, basandosi su quattro semplici regole:

- **Morte per solitudine:** Una cella viva muore se ha meno di due vicini vivi.
- **Sopravvivenza:** Una cella viva con due o tre vicini vivi sopravvive alla generazione successiva.
- **Morte per sovraffollamento:** Una cella viva muore se ha più di tre vicini vivi.
- **Riproduzione:** Una cella morta diventa viva se ha esattamente tre vicini vivi.

Il flusso di esecuzione del problema è il seguente:

1. **Calcolo Locale:** Per ogni cella (x, y) , calcolare il numero di vicini vivi $V(x, y)$.
2. **Applicazione Regole:** Applicare le regole di transizione per determinare il nuovo stato $S_{t+1}(x, y)$.
3. **Aggiornamento:** Scrivere il nuovo stato in una griglia di gioco $T+1$ di transizione temporanea, in modo da evitare che gli aggiornamenti influenzino i calcoli dei vicini nella stessa generazione.
4. **Scambio:** la griglia di gioco $T+1$ diventa la griglia T per la generazione successiva.

1.1 Identificazione delle dipendenze tra dati

- **Dipendenze temporali (Inter-generazione):** la gen. $T+1$ è totalmente dipendente dalla gen. T . Questo fa sì che la simulazione sia intrinsecamente sequenziale nel tempo, ovvero il calcolo della generazione $T+2$ non può iniziare fino a che $T+1$ non è stata completamente terminata.
- **Dipendenze spaziali (Intra-generazione):** il calcolo del nuovo stato di una cella (x, y) dipende solo ed esclusivamente dallo stato dei vicini a raggio 1 (8 celle adiacenti), mentre è completamente indipendente da quello di una qualsiasi altra cella (x', y') che non rientri nel suo vicinato.

Da queste considerazioni si può dedurre che il GoL è un problema che soffre principalmente di dipendenze sequenziali nel tempo, mentre quelle spaziali sono limitate alla località delle celle. Proprio per questi motivi il GoL si presenta come un ottimo problema su cui applicare la parallelizzazione.

Tutte le analisi presentate in questa relazione sono state realizzate attraverso estrapolazione di statistiche di performance eseguendo i codici consegnati nei runtime offerti da Google Colab. Inoltre, per l'implementazione tramite CUDA, ci si è avvalsi degli strumenti di analisi Nsight Compute e Nsight System.

2. Implementazione Sequenziale

L'implementazione sequenziale base si concentra al massimo sulla correttezza funzionale e sulla semplicità di realizzazione, elaborando la griglia di gioco con due cicli annidati: riga per riga e colonna per colonna. Lo stato delle celle è rappresentato dal tipo `char` di un byte. Per quanto riguarda le celle appartenenti alla prima riga/colonna e ultima riga/colonna, la scelta è stata quella di considerare i loro vicini mancanti come celle morte (*zero-padding*), aggiungendo due righe e due colonne di zeri alla griglia logica.

La scelta dell'uso del `char` (rispetto a 1bit per cella) è stata dettata da esigenze di efficienza nell'accesso alla memoria e nella manipolazione dei dati: le moderne CPU sono *byte-addressable* (l'unità di memoria più piccola indirizzabile direttamente è il byte), in questo modo la cella è già allineata e consente scritture e letture dirette; il conteggio dei vicini funziona come una somma diretta. Quindi, anche se utilizzando un array di bit si ridurrebbe il consumo di memoria di 8x, l'overhead computazionale introdotto per l'accesso ai bit e la gestione della logica (operazioni bit-wise per isolare i singoli bit) è generalmente maggiore di quello risparmiato nel caricamento dei dati.

Questa semplice versione ci serve come riferimento per misurare le prestazioni e lo speed-up rispetto alle versioni SIMD e parallela tramite CUDA.

3. Implementazione SIMD

Questa versione è basata su istruzioni SSE (*Streaming SIMD Extensions*) per eseguire la logica del GoL utilizzando registri estesi a 128bit. Si basa su determinati principi di base:

- **Vettorizzazione:** il calcolo passa da cella per cella a blocchi di 16 celle contigue, poiché ogni `char` è rappresentato da 1 byte e i registri SSE sono `__m128i` = 128 bit (16 byte).
- **Allineamento:** allochiamo la memoria con allineamento a 16 byte (`aligned_malloc_grid()`) per ottenere le massime prestazioni SIMD e un accesso allineato alla memoria.
- **Overlapping Load:** per il calcolo dei vicini di 16 celle centrali abbiamo la necessità di caricare 3 blocchi di 16 byte per ogni riga (precedente, corrente, successiva).

3.1 Caricamento dei dati

Nel ciclo interno sulle colonne (`j += 16`), carichiamo 9 vettori di 16 byte ciascuno contenenti i vicini e la cella corrente di tutte e 16 le celle contenute in un registro esteso.

```
// Caricamenti sovrapposti per la riga precedente
__m128i prev_left = _mm_loadu_si128((__m128i*)(row_prev + j - 1)); // j-1 .. j+14
__m128i prev_mid = _mm_loadu_si128((__m128i*)(row_prev + j)); // j .. j+15
__m128i prev_right = _mm_loadu_si128((__m128i*)(row_prev + j + 1)); // j+1 .. j+16

// Caricamenti sovrapposti per la riga corrente
__m128i cur_left = _mm_loadu_si128((__m128i*)(row_curr + j - 1));
__m128i cur_mid = _mm_loadu_si128((__m128i*)(row_curr + j));
__m128i cur_right = _mm_loadu_si128((__m128i*)(row_curr + j + 1));

// Caricamenti sovrapposti per la riga successiva
__m128i next_left = _mm_loadu_si128((__m128i*)(row_next + j - 1));
__m128i next_mid = _mm_loadu_si128((__m128i*)(row_next + j));
__m128i next_right = _mm_loadu_si128((__m128i*)(row_next + j + 1));
```

Andiamo ad utilizzare l'istruzione `_mm_loadu_si128()` per gestire i caricamenti non allineati: i vicini *left* e *right* non iniziano perfettamente sul confine di 16 byte.

3.2 Calcolo vettoriale dei vicini

```
// Calcolo dei vicini vivi
__m128i neigh_alive = _mm_add_epi8(prev_left, cur_left);
neigh_alive = _mm_add_epi8(neigh_alive, next_left);
neigh_alive = _mm_add_epi8(neigh_alive, prev_mid);
neigh_alive = _mm_add_epi8(neigh_alive, next_mid);
neigh_alive = _mm_add_epi8(neigh_alive, prev_right);
neigh_alive = _mm_add_epi8(neigh_alive, cur_right);
neigh_alive = _mm_add_epi8(neigh_alive, next_right);
```

Per ottenere la somma dei vicini usiamo l'istruzione vettoriale di somma `_mm_add_epi8` per sommare byte per byte gli 8 vettori di vicini. In questo modo ogni byte del registro `neigh_alive` conterrà il conteggio dei vicini per la cella corrispondente.

3.3 Applicazione vettoriale delle regole

Per poter applicare le regole contemporaneamente a tutte le 16 celle usiamo istruzioni di confronto e operazioni logiche bit-wise vettoriali, sfruttando le maschere `three_vec`, `two_vec`:

```
// Regola 1: Nuova vita o Sopravvivenza (N=3):
// Crea una maschera dove i byte sono 0xFF (vero) se il conteggio dei vicini è 3.
__m128i is_three = _mm_cmpeq_epi8(neigh_alive, three_vec);
// Regola 2: Sopravvivenza (N=2):
// Crea una maschera dove i byte sono 0xFF se il conteggio è 2.
__m128i is_two = _mm_cmpeq_epi8(neigh_alive, two_vec);
// Combinazione delle Regole: La maschera dello stato futuro è:
// (N=3) OR (Stato Attuale AND N=2). La cella attuale è contenuta nel vettore centrale della riga corrente (cur_mid).
__m128i new_state_mask = _mm_or_si128(
    is_three,
    _mm_and_si128(cur_mid, is_two)
);
```

3.4 Conversione e memorizzazione

```
// Conversione in Binario: Le maschere di confronto contengono 0xFF per "vero" e 0x00 per "falso".
// Per ottenere lo stato finale come 1 o 0, si esegue un AND logico
// con il vettore costante one_vec (contenente 0x01 per ogni byte):
_mm128i next_cells = _mm_and_si128(new_state_mask, one_vec);

// Memorizzazione del nuovo stato della griglia di gioco
char* target_ptr_next = next_grid + i * PADDED_SIZE + j;
_mm_storeu_si128((__m128i*)target_ptr_next, next_cells);
```

3.5 Confronto e analisi tramite statistiche

Abbiamo realizzato una serie di statistiche di performance (tabella seguente) che confrontano le due implementazioni Seq vs. SIMD (SSE) per una griglia di gioco 2048x2048 per 15 generazioni, sotto diversi livelli di ottimizzazione del compilatore ($-Ox$) ($-O0$ =nessuna $\rightarrow$ $-O3$ =aggressiva). Questo flag indica al compilatore di applicare una serie di trasformazioni al codice generato per migliorarne le prestazioni, bilanciando velocità di esecuzione, dimensione del codice e tempo di compilazione.

Le analisi sono state svolte distinguendo tra:

- **Tempo Totale:** include inizializzazione, allocazione/deallocazione della memoria e calcolo delle generazioni.
- **Tempo Generazioni:** misura solo il tempo speso per la logica di aggiornamento di tutte le generazioni.

3.5.1 Analisi del tempo delle generazioni

FLAG	TEMPO SEQ.	TEMPO SIMD	SPEED-UP	EFFICIENZA	OSSERVAZIONI
-O0	745,42 ms	191,49 ms	3,89x	24,33%	Basso Speed-up: logica sequenziale non ottimizzata
-O1	184,99 ms	14,85 ms	12,46x	77,86%	Il miglioramento di 4x nel sequenziale e l'efficacia del SIMD portano allo Speed-up ottimale.
-O2	178,59 ms	14,42 ms	12,38x	77,38%	Prestazioni quasi identiche a -O1. Massima efficacia operativa del codice SIMD.
-O3	56,61 ms	14,79 ms	3,83x	23,93%	Forte auto vettorizzazione del sequenziale, che riduce drasticamente lo Speed-up.
-O4	66,42 ms	14,45 ms	4,60x	28,72%	Tempo SIMD più basso tra le ottimizzazioni aggressive.
-O5	57,55 ms	18,91 ms	3,04x	19,02%	Anomalia: il tempo SIMD aumenta
-O6	59,38 ms	18,81 ms	3,16x	19,73%	Simile a -O5

Griglia di gioco: 2048x2048

Generazioni=15

Da questi dati possiamo trarre una serie di conclusioni:

- **Stabilità Tempo SIMD** a partire da -O1: questo indica come il codice SSE esplicito sia altamente ottimizzato e poco influenzato dai livelli di ottimizzazione successivi.
- **Impatto dell'ottimizzazione sul codice sequenziale:** da -O0 a -O1 il Tempo_seq è ridotto di ~4.1 volte, da -O2 a -O3 di altre ~3 volte. Questo è causato dall'attivazione della vettorizzazione automatica del compilatore o da ottimizzazioni aggressive dei loop, causando anche un conseguente crollo dello Speed-up da -O3 che passa da ~12.47x a ~4.04x. Questo fa sì che abbiamo anche efficienza massima a -O1 e -O2 (~78%).
- **Anomalie a -O5 e -O6:** il tempo SIMD aumenta inspiegabilmente a ~18-19 ms. Questo comportamento potrebbe suggerire che le ottimizzazioni aggressive interferiscono negativamente con gli *intrinsics* SSE espliciti, compromettendo l'efficacia della gestione dei registri o l'allineamento dei dati.

Per questi motivi, nelle analisi successive limitiamo le ottimizzazioni fino a -O3.

3.5.2 Analisi del tempo totale

FLAG	T. SEQUENZIALE	T. SIMD	SPEED-UP TOTALE	SPEED-UP GENERAZIONI
-O0	844,28 ms	288,93 ms	2,92x	3,89x
-O1	279,52 ms	106,42 ms	2,63x	12,46x
-O2	274,66 ms	108,38 ms	2,53x	12,38x
-O3	148,72 ms	106,91 ms	1,39x	3,83x

Griglia di gioco: 2048x2048 Generazioni=15

Vediamo come lo Speed-up Totale sia sempre molto più basso di quello delle Generazioni, in particolare notiamo una forte riduzione ad -O3 (3.83x -> 1.39x). Questa è data dal fatto che l'overhead di setup (~92 - 98 ms in entrambi) non può essere parallelizzato e quindi diventa una parte molto più grande rispetto al tempo di calcolo delle generazioni, in accordo con la Legge di Amdahl.

3.5.3 Analisi per dimensioni e generazioni maggiori

In seguito, riportiamo alcune metriche relative a griglia di gioco e a generazioni maggiori.

FLAG	METRICA	2048x2048 (15 GEN)	32.768x32.768 (100 GEN)
-O0	Speed-up Generazioni	3,89x	3,98x
	Efficienza Generazioni	24,33%	24,85%
-O1/-O2	Speed-up Generazioni	~ 12,4x	~ 10,87x
	Efficienza Generazioni	~ 77,8%	~ 67,9%
	Overhead setup seriale	~ 96 ms	~ 24.000 ms
	Speed-up Totale massimo		6,63x
-O3	Speed-up Gen.	3,83x	3,55x

Come si nota dalla tabella per la griglia grande, lo Speed-up si abbassa a ~ 10,8x e, di conseguenza, l'efficienza cala a ~ 67,9%. Questo è dovuto al fatto che il problema è passato ad essere *bandwidth-bound*: la griglia è troppo grande per stare completamente in cache-CPU e i dati devono essere costantemente trasferiti tra RAM e CPU. In accordo alla Legge di Amdahl è chiaro come l'overhead seriale di setup limiti lo speed-up totale SIMD del problema a 6,63x, rispetto a quello del calcolo delle generazioni ~ 10,87x.

4. Decomposizione e Struttura Parallela CUDA

Il principio di base del Game of Life si adatta perfettamente al modello di programmazione **Thread Level Parallelism**, che assegna un thread GPU per ogni sotto-problema, nel nostro caso ogni thread è dedicato ad ogni singola cella della griglia di gioco.

Il problema del GoL soffre però di limitazioni o criticità intrinseche, come:

- **Gestione dei vicini per le celle sui bordi:** nella nostra soluzione parallela tramite CUDA seguiremo la stessa idea usata nell'implementazione SIMD di una griglia di gioco NxN con Zero Padding. Questo richiederà una logica lievemente più complessa nel kernel per i thread assegnati a celle di bordo.
- **Overhead di allocazione e trasferimento dati (Host CPU <-> Device GPU):** il tempo speso per i trasferimenti della griglia di gioco e l'allocazione della memoria potrebbero essere maggiori del tempo di calcolo effettivo sulla GPU per griglie di gioco piccole. La GPU risulterà vantaggiosa solo se usata per gestire **griglie grandi** e/o calcolare un **elevato numero di generazioni**. Da questo si può prevedere come il GoL sia migliore in CPU per griglie piccole e migliore in GPU per griglie grandi.

- **Barriera di sincronizzazione temporale:** tutti i thread devono aver completato il loro calcolo sulla generazione attuale prima che la successiva possa iniziare. Questo implicherà l'uso di una barriera di sincronizzazione globale tra le generazioni con `cudaDeviceSynchronize()`, andando ad utilizzare anche due griglie (una di lettura e una di scrittura) per evitare il verificarsi di *race conditions*. La decomposizione temporale sarà dunque di tipo sequenziale: un loop CPU avvia un kernel per ogni generazione, questo calcola il vicinato e lo stato futuro di ogni cella, utilizzando la Global Memory della GPU per salvare le griglie di Input e Output.

Fatte queste considerazioni di seguito andiamo a presentare diverse strategie di decomposizione parallela del problema del Game of Life tramite GPU e CUDA.

5. CUDA – Ottimizzazioni incrementali

5.1 Implementazione Naïve

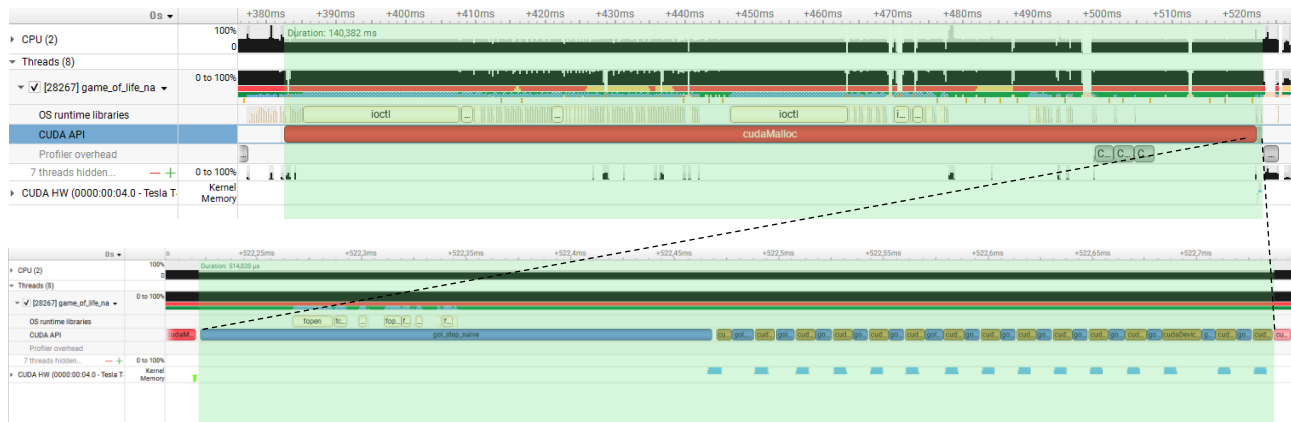
La prima implementazione parallela tramite GPU e CUDA è stata realizzata tramite tecniche base e naïve di parallelizzazione. La griglia del kernel è stata dimensionata come vettore 1D con un singolo blocco (dimGriglia = 1). Questo contiene il numero massimo di thread per blocco, corrispondente a 1024 (dim blocco = (32,32,1)).

La griglia di gioco, di conseguenza, è una matrice 32x32. Come test, sono state simulate 15 generazioni.

	T. ESEC. TOTALE	T. GENERAZIONI	SPEED-UP GENERAZIONI	SPEED-UP SUL TOTALE
SEQUENZIALE	0,2699 ms	236,13 μ s		
SIMD (-O0)	0,1243 ms	65,263 μ s	3,68x	2,17x
SIMD (-O1)	0,0291 ms	4,116 μ s	15,9x	3,74x
CUDA NAÏVE	140,38 ms	514,82 μ s	N/C	N/C

Griglia di gioco: 32x32 Blocco: 32x32 Generazioni=15

La tabella sopra mostra come il tempo di esecuzione totale per il caso CUDA naïve sia di molto maggiore rispetto al caso sequenziale, un po' meno per il calcolo delle 15 generazioni. Questo risultato è del tutto prevedibile e comprensibile osservando il diagramma temporale riportato di seguito ottenuto tramite Nsight System.



Il programma impiega la maggior parte del tempo per l'allocazione della memoria iniziale (`cudaMalloc()`) e il trasferimento dei dati nella GPU (`cudaMemcpy()`), incrementando vertiginosamente il tempo di esecuzione. Per una griglia di gioco così piccola e per il numero limitato di generazioni, questi contributi non sono trascurabili affatto. Osservando infatti il tempo di esecuzione in riferimento al calcolo delle generazioni da parte dei kernel e dello stallo della CPU in attesa dei loro completamenti (`cudaDeviceSynchronize()`), notiamo come siano una minima frazione del tempo totale.

Sebbene grazie a questa configurazione gli indici del thread del blocco (`threadIdx.x` e `threadIdx.y`) corrispondano alle coordinate delle celle della griglia di gioco, questa soluzione presenta chiari ed evidenti limiti operativi e inefficienze nell'uso delle risorse:

1. **Massima dimensione della griglia:** disponendo di un limite fisico per numero di thread per blocco, la griglia di gioco risulta essere sottoposta allo stesso limite. Il massimo numero di celle che questa versione può supportare è proprio 1024 => griglia di gioco 32x32.
2. **Impiego minimo degli SM:** Un blocco può essere assegnato ad un solo SM. Poiché questa configurazione presenta solo un blocco, solo uno degli SM a disposizione è interessato dall'esecuzione dell'intero kernel, mantenendo inattivi i restanti (per Tesla T4: 1 SM attivo su 40 a disposizione).

Questi sono solo due dei principali limiti relativi a questa soluzione. Come primo passaggio ci siamo focalizzati sulla loro risoluzione, considerando altre problematiche relative a utilizzo della shared memory, accessi ottimizzati in memoria e warp divergence in versioni di ottimizzazione successive.

Per prima cosa, quindi, abbiamo modificato il dimensionamento della griglia e dei blocchi di thread nella maniera descritta al paragrafo successivo.

5.2 Implementazione tramite Griglia 2D e Blocco 2D

In questa versione abbiamo mantenuto la dimensione del blocco pari a 1024 thread (32x32), ma abbiamo trattato anche la griglia in 2D. In particolare, la sua dimensione è ottenuta in relazione a:

1. Dimensione della griglia di gioco, espressa in *width* e *height*.
2. Dimensione dei blocchi, ovvero numero di thread per blocco.

```
dim3 dimGrid(
    (width + dimBlock.x - 1) / dimBlock.x,
    (height + dimBlock.y - 1) / dimBlock.y
);
```

Grazie a questo dimensionamento non abbiamo più vincoli sulla dimensione massima della griglia di gioco, ma solo limiti dovuti alle risorse hardware della GPU in esame. Abbiamo quindi considerato una griglia di gioco 1024x1024 (1.048.576 celle) e mantenuto il numero di generazioni pari a 15. La tabella successiva ne mostra i risultati.

	T. ESEC TOTALE	T. GENERAZIONI	SPEED-UP GENERAZIONI	SPEED-UP SUL TOTALE
SEQUENZIALE	210,9 ms	184,8 ms		
SIMD (-O0)	71,26 ms	47,02 ms	3,9x	2,96x
SIMD (-O1)	35,28 ms	5,06 ms	15,2x	3,13x
CUDA 32X32	132,3 ms	2,49 ms	74,0x	1,59x
Griglia di gioco: 1024x1024 Blocco: 32x32 Generazioni=15				

Grazie all'approccio tramite CUDA, lo speed-up in relazione al solo calcolo delle generazioni presenta risultati migliori al caso SIMD. Tuttavia, in relazione allo speed-up sul tempo totale, questa implementazione non è ancora in grado di battere la versione SIMD -O0: l'overhead dell'allocazione e delle barriere di sincronizzazione tra CPU e GPU hanno ancora un impatto forte.

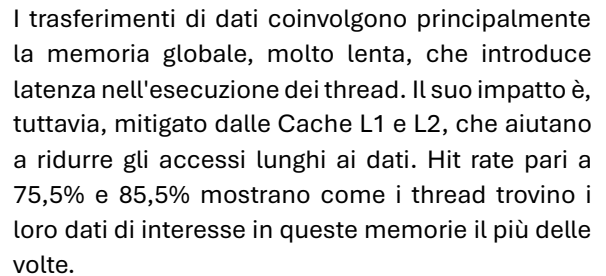
5.2.1 Analisi tramite Nsight Compute

Analizzando l'esecuzione del kernel tramite lo strumento Nsight Compute abbiamo notato le seguenti criticità:

Compute (SM) Throughput [%]	51,49	Duration [us]	133,02
Memory Throughput [%]	27,41	Elapsed Cycles [cycle]	77.808
L1/TEX Cache Throughput [%]	29,91	SM Active Cycles [cycle]	71.185,93
L2 Cache Throughput [%]	3,77	SM Frequency [Mhz]	584,89
DRAM Throughput [%]	5,61	DRAM Frequency [Ghz]	4,98

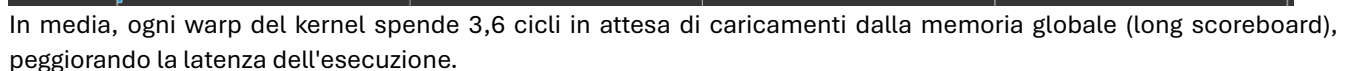
Il kernel mostra un *Compute Throughput* basso, pari al 51,5%, e un *Memory Throughput* ancora inferiore, pari al 27,4% rispetto ai valori di peak performance. Questi risultati confermano quello che ci si poteva aspettare: in questa versione preliminare non ci si è preoccupati della gestione degli accessi in memoria in maniera curata, né dell'uso di memoria on-chip come la *shared memory*.

Memory Throughput [Gbyte/s]	17,90	Mem Busy [%]	14,06
L1/TEX Hit Rate [%]	75,52	Max Bandwidth [%]	27,41
L2 Hit Rate [%]	85,49	Mem Pipes Busy [%]	27,41



Considerando le sezioni **Occupancy** e **Warp State Statistics**:

Notiamo come, nonostante l'occupancy effettiva sia abbastanza alta (78,3%), raggiungiamo un Compute Throughput del 51% circa. Questo evidenzia come gli SM siano carichi di warp attivi, ma con thread *in stallo*. Il dato *Warp cycles per issued instruction* pari a 11,06 cicli è abbastanza esplicativo: servono 11 cicli per l'emissione di 2 istruzioni successive dello stesso warp. Il seguente grafico sullo stato del warp conferma questa situazione:



Avg. Active Threads Per Warp	31,97
Avg. Not Predicated Off Threads Per Warp	28,52

Il secondo dato, però, evidenzia come in media solo 28,52 di questi thread stia eseguendo l'istruzione corrente, contro 3,45 thread che sono inattivi perchè nel ramo opposto di un if-else.

La conferma di ciò è data anche dalla sezione **Source Counters**:

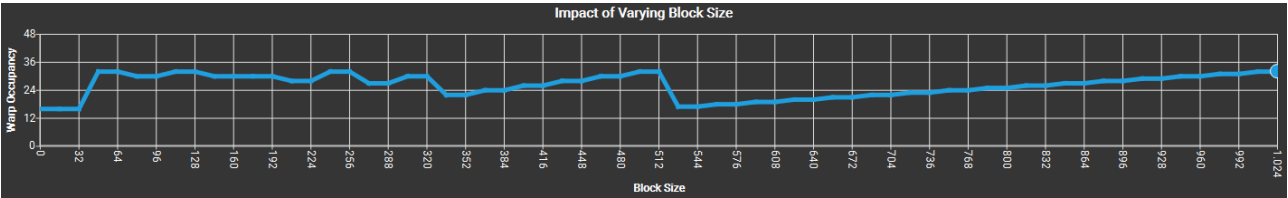
Branch Instructions [inst]	1.185.788	Branch Efficiency [%]	99,25
Branch Instructions Ratio [%]	0,19	Avg. Divergent Branches [branches]	38,38

Il valore *Avg. Divergent Branches* indica come in media ogni warp incontra 38,38 rami divergenti nella sua esecuzione. Questi rami sono dovuti soprattutto all'accesso alle celle di gioco sui bordi che non eseguono l'if per determinare il valore della cella vicina appartenente allo zero-padding. Meno probabilmente appartengono al ramo if che determina se la cella considerata appartiene alla griglia di gioco o no, dal momento che per questa implementazione il numero di thread coincide con il numero di celle di gioco, escludendo la presenza di thread di "riempimento" di blocco inattivi perché non associati ad alcuna cella.

5.2.2 Ottimizzazioni future

In base alle analisi precedenti, è evidente come il vero collo di bottiglia sia rappresentato dalle lunghe attese di accessi alla memoria globale, anche se le cache L1 e L2 svolgono un ruolo importante nella loro mitigazione. Possiamo affermare che, vista la semplicità dei calcoli determinanti gli stati della nuova griglia di gioco, il problema in esame sia *memory-bound* piuttosto che *compute-bound*. Per ottenere migliori prestazioni nelle successive implementazioni, quindi, si dovrà ottimizzare l'accesso alla memoria. Tuttavia, limitatamente alla prossima implementazione, il nostro obiettivo è stato quello di identificare la strada migliore per ottimizzare l'*occupancy* effettiva dei warp, in modo da sfruttare al meglio le risorse.

Come mostra il grafico sull'impatto della variazione della dimensione del blocco, considerando 256 thread per blocco (16x16) la warp occupancy teorica viene mantenuta al 100%. Di conseguenza, ci aspettiamo di riscontrare una occupancy effettiva maggiore, disponendo ora i warp scheduler di più warp da poter interscambiare per aumentare il *latency hiding*.



Per investigare l'impatto di questa modifica, quindi, la prossima versione presenterà:

- 1. Modifica della dimensione dei blocchi a 16x16.
- 2. Risoluzione della warp divergence per determinare il valore della cella tramite operatori booleani.

Rimandiamo ottimizzazioni riguardo la memoria e i suoi accessi ad una futura ottimizzazione.

5.3 Ottimizzazione con blocco 16x16

In base alle analisi della soluzione precedente, abbiamo ridotto la dimensione del blocco a 16x16 ed eliminata la warp divergence relativa all'aggiornamento del valore della nuova cella. La tabella sottostante mostra i risultati ottenuti.

	T. ESEC TOTALE	T. GENERAZIONI	SPEED-UP GENERAZIONI	SPEED-UP SUL TOTALE
SEQUENZIALE	210,91 ms	184,85 ms		
SIMD (-O0)	71,26 ms	47,02 ms	3,9x	2,96x
SIMD (-O1)	35,28 ms	05,06 ms	15,2x	3,13x
CUDA 32X32	132,34 ms	2,497 ms	74,0x	1,59x
CUDA 16X16	105,20 ms	1,980 ms	93,3x	2,00x

Griglia di gioco: 1024x1024 Blocco: 16x16 Generazioni=15

Rispetto alla soluzione precedente, abbiamo ottenuto un aumento delle prestazioni sia sul tempo di esecuzione totale, che dei 15 kernel con conseguente aumento dello speed-up. Avvalendoci sempre dello strumento Nsight Compute, abbiamo analizzato i punti di forza di questo miglioramento:

Compute (SM) Throughput [%]	61,81	Duration [us]	109,92
Memory Throughput [%]	33,10	Elapsed Cycles [cycle]	64.299
L1/TEX Cache Throughput [%]	62,29	SM Active Cycles [cycle]	62.491,32
L2 Cache Throughput [%]	7,95	SM Frequency [Mhz]	584,92
DRAM Throughput [%]	14,11	DRAM Frequency [ghz]	4,98

I dati mostrano come sia stato ottenuto un incremento non solo del *Compute Throughput*, da 51,56% al 61,81%, ma anche del *Memory Throughput*, dal 27,44% al 33,10%.

Considerando la sezione **Compute Workload Analysis**, infatti,

Executed Ipc Elapsed [inst/cycle]	2,47	SM Busy [%]	63,65
Executed Ipc Active [inst/cycle]	2,54	Issue Slots Busy [%]	63,65
Issued Ipc Active [inst/cycle]	2,55		

notiamo un incremento anche della metrica SM Busy, con un aumento di 7,25 punti percentuali. Questo evidenzia come le ALU (essendo il problema in esame interessato solo da operazioni tra interi) siano meglio utilizzate rispetto al caso precedente.

Warp State Statistics

Warp Cycles Per Issued Instruction [cycle]	9,56	Avg. Active Threads Per Warp	31,98
Warp Cycles Per Executed Instruction [cycle]	9,56	Avg. Not Predicated Off Threads Per Warp	28,52

La tabella soprastante indica come si siano mantenute stabili le metriche rispetto alle medie di *thread attivi per warp* e *warp che seguono lo stesso ramo condizionale*, indice di come le divergenze non fossero dettate dal ramo condizionale relativo al valore della cella (eliminato in questa versione), ma come dipendano dagli accessi ai vicini "zero-padded" per le celle ai bordi della griglia. Il numero totale di istruzioni di branch (nella sezione **Source Counters**) infatti passa da 1.185.788 a 1.183.100.

Inoltre, si è ottenuto un miglioramento sul numero di cicli di clock necessari per l'emissione di due istruzioni successive appartenenti allo stesso warp: da 11,05 precedenti a 9,56 attuali.

5.3.1 Analisi dei risultati ottenuti

I miglioramenti esposti precedentemente sono stati ottenuti grazie ad un fattore principale: la maggiore granularità dei blocchi.

Occupancy

Theoretical Occupancy [%]	100	Block Limit Registers [block]	8
Theoretical Active Warps per SM [warp]	32	Block Limit Shared Mem [block]	16
Achieved Occupancy [%]	76,75	Block Limit Warps [block]	4
Achieved Active Warps Per SM [warp]	24,56	Block Limit SM [block]	16

Analizzando, infatti, la sezione **Occupancy** possiamo osservare come il numero di blocchi limite per SM sia passato da 1 a 4. Questo permette ai warp scheduler di poter effettuare *latency-hiding* interoperando i 4 blocchi a loro disposizione nel momento in cui uno di questi dovesse essere bloccato da istruzioni lunghe (nel nostro caso lunghe attese di lettura da memoria globale). In questo modo, sia le ALU che il bus di memoria saranno più "densamente" coinvolti.

Queste considerazioni spiegano gli aumenti di Compute Throughput, Memory Throughput e il minor numero di cicli di clock per l'emissione di due istruzioni riportati precedentemente.

In contrasto a quanto avevamo ipotizzato prima dell'implementazione di questa versione, il dato sulla *Achieved Occupancy* non risulta migliorato, ma di poco inferiore al caso precedente (78,32 -> 76,75). Trattandosi questa di una misura sul totale (percentuale) questo abbassamento è sintomo di un fattore principale: non tutti i blocchi assegnati allo stesso SM terminano nello stesso istante. Nel tempo in cui il warp scheduler assegna un altro blocco allo SM, l'occupancy scende dal 100% al 75%, abbassando questa statistica.

5.3.2 Considerazioni sull'hit rate di Cache L2

Memory Workload Analysis

Memory Throughput [Gbyte/s]	45,00	Mem Busy [%]	31,15
L1/TEX Hit Rate [%]	76,13	Max Bandwidth [%]	33,10
L2 Hit Rate [%]	74,66	Mem Pipes Busy [%]	33,10

Osservando bene le metriche relative al workload della memoria, possiamo notare un peggioramento nell'hit rate per la cache L2 da 85,49% a 74,66%. È vero che il throughput di memoria è incrementato (da 18.07 a 45 GB/s) come effetto dell'aumento dei blocchi, ma gli accessi non coalescenti sono peggiorati. Come dettaglia il report, in questa configurazione solo 12,4 dei 32 byte trasmessi per settore sono utilizzati, contro i 20,2 della configurazione precedente. Ciò è dovuto alla natura del problema: avendo blocchi più piccoli, ogni blocco gestisce meno celle. Poiché non possiamo decidere l'assegnamento dei blocchi agli SM, blocchi contigui della griglia di gioco possono essere assegnati a SM differenti. Questo comporta che thread di un blocco carichino celle utili al blocco contiguo ma che, non trovandosi sullo stesso SM, non possano usufruire di questi dati appena caricati in cache L1. Questo presuppone che il blocco contiguo debba rivolgersi alla cache L2, la quale però, a causa del flusso di dati maggiore

generato da più blocchi (45 GB/s), è soggetta ad un maggiore tasso di *eviction* (sovrascrittura) costringendo il blocco a rivolgersi alla lenta memoria globale.

5.3.3 Ottimizzazioni future

In conclusione, possiamo affermare che il vero collo di bottiglia rimasto sia rappresentato dall'inefficiente gestione degli accessi alla memoria in maniera non coalescente e dal mancato uso della memoria condivisa. Viste le considerazioni di cui sopra, la successiva ottimizzazione si concentrerà sugli accessi efficienti in memoria e sull'utilizzo della shared memory.

5.4 Ottimizzazione con Shared Memory

In questa versione di ottimizzazione ci siamo focalizzati sull'uso efficiente della memoria utilizzando la shared memory come principale fonte di accesso ai dati.

Il kernel è stato modificato inserendo una variabile `__shared__ u8 tile[SM_h][SM_w]` per l'allocazione statica della griglia di gioco relativa al blocco in questione:

- `SM_h`: altezza pari alla dimensione y del blocco, più due righe per permettere di caricare i vicini delle celle di gioco ai bordi. Questi, infatti, devono accedere alle celle appartenenti ad altri blocchi, o valori posti a zero (zero-padding) nel caso di celle appartenenti alla prima/ultima riga/colonna della griglia di gioco.
- `SM_w`: larghezza pari alla dimensione x del blocco, più due colonne per accesso ai vicini come descritto prima.

Ogni thread, quindi, caricherà la propria cella di interesse e i vicini nella shared memory. È fondamentale qui imporre una barriera di sincronizzazione tramite `__syncthreads()` per assicurarsi che tutti i thread del blocco abbiano caricato i propri dati rendendoli disponibili agli altri thread del blocco.

Una volta raggiunto questo punto, l'accesso successivo a questi dati sarà agevolato grazie alla velocità della memoria condivisa.

	T. ESEC. TOTALE	T. GENERAZIONI	SPEED-UP GENERAZIONI	SPEED-UP SUL TOTALE
SEQUENZIALE	210,91 ms	184,85 ms		
SIMD (-O0)	71,26 ms	47,02 ms	3,9x	2,96x
SIMD (-O1)	35,28 ms	05,06 ms	15,2x	3,13x
CUDA 32X32	132,34 ms	2,497 ms	74,0x	1,59x
CUDA 16X16	105,20 ms	1,980 ms	93,3x	2,00x
CUDA SMEM	104,19 ms	1,945 ms	95,0x	2,02x

Griglia di gioco: 1024x1024 Blocco: 16x16_SharedMemory Generazioni=15

Come mostra la tabella, il tempo di esecuzione non è calato drasticamente, ma analizzando meglio i dati tramite *Nsight Compute* notiamo molti miglioramenti:

GPU Speed of Light Throughput

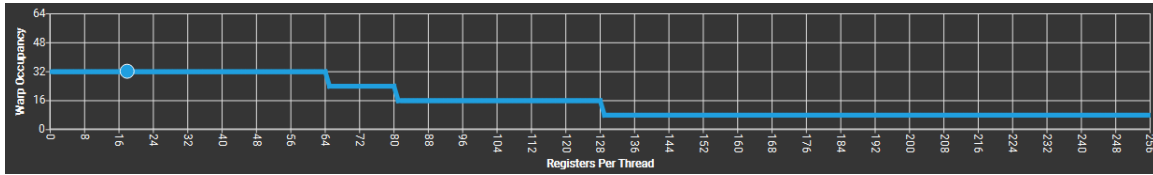
Compute (SM) Throughput [%]	58,67	Duration [us]	108,51
Memory Throughput [%]	58,67	Elapsed Cycles [cycle]	63.441
L1/TEX Cache Throughput [%]	70,01	SM Active Cycles [cycle]	61.509,40
L2 Cache Throughput [%]	9,25	SM Frequency [Mhz]	584,58
DRAM Throughput [%]	14,44	DRAM Frequency [Ghz]	4,97

Rispetto al caso precedente, Il throughput di memoria è salito ad un totale di 58,67% da 33,22%. Questo è dovuto ad un caricamento collaborativo in shared memory da parte dei thread di un warp, aumentando l'utilizzo effettivo dei byte trasferiti.

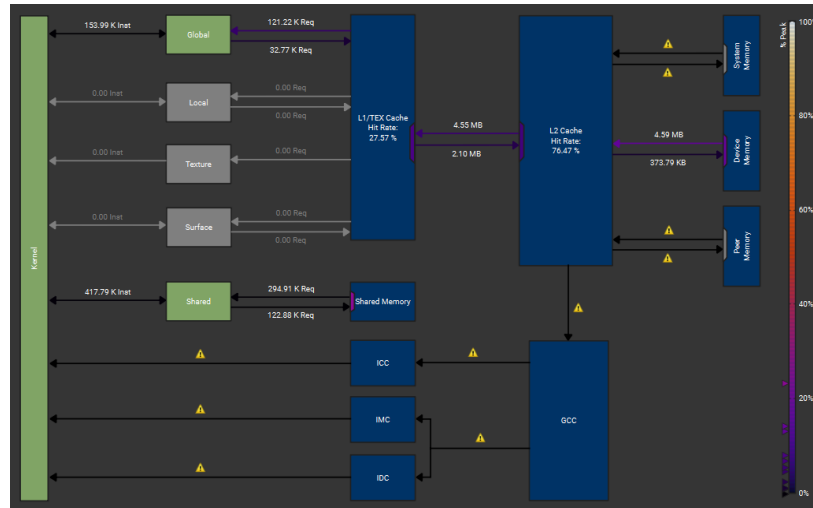
Occupancy

Theoretical Occupancy [%]	100	Block Limit Registers [block]	10
Theoretical Active Warps per SM [warp]	32	Block Limit Shared Mem [block]	64
Achieved Occupancy [%]	91,32	Block Limit Warps [block]	4
Achieved Active Warps Per SM [warp]	29,22	Block Limit SM [block]	16

L'occupancy effettiva risulta essere aumentata al 91,3% dal 76,8% precedente. Infatti, anche i warp attivi per SM raggiunti sono aumentati a 29,2 da 24,6. Questi miglioramenti sono dovuti all'utilizzo della shared memory e una conseguente pressione inferiore sui registri dello SM (24->18) come è possibile vedere nel seguente grafico:



Memory Workload Analysis



Analizzando il grafico soprastante può risultare che il forte abbassamento dell'hit rate per la cache L1/tex (27.57%) sia sintomo di un peggioramento (considerando il precedente hit rate pari a 78.27%). In realtà è il risultato che ci aspettavamo: ogni thread si rivolge alla cache L1 principalmente per i caricamenti iniziali delle proprie celle di interesse che, non essendo disponibili, portano a rivolgersi alle memorie successive abbassandone l'hit rate. Una volta caricati i dati, quindi, ogni thread utilizzerà esclusivamente la memoria condivisa per accedere ai vicini per il calcolo dell'aggiornamento della propria cella. Questo è confermato dal maggior numero di richieste di accesso alla Shared Memory (294.91 K in lettura) rispetto a quelle che interessano la cache L1 (121.22 K).

Instruction Statistics

Executed Instructions [inst]	3,788,170	Avg. Executed Instructions Per Scheduler [inst]	23,676,06
Issued Instructions [inst]	3,793,530	Avg. Issued Instructions Per Scheduler [inst]	23,709,56

Un ulteriore traguardo raggiunto dall'implementazione tramite shared memory risiede nel numero totale di istruzioni eseguite: otteniamo un calo pari a circa il 40% da 6.360.444 senza memoria condivisa alle attuali 3.788.170. Il caricamento collaborativo di dati dalla memoria riduce notevolmente il numero di istruzioni necessarie al kernel.

Warp State Statistics

Warp Cycles Per Issued Instruction [cycle]	18,73	Avg. Active Threads Per Warp	26,22
Warp Cycles Per Executed Instruction [cycle]	18,75	Avg. Not Predicated Off Threads Per Warp	25,45

Questo calo di istruzioni è il motivo per cui, dall'altra parte, osserviamo un considerevole aumento del *Warp Cycle per Issued Instruction (WCPI)* pari a 18,73 (prima 9,56). Il numero di istruzioni è stato dimezzato, ma i cicli totali spesi per attendere i dati dalla DRAM sono rimasti costanti, innalzando questo valore. La GPU passa più tempo ad aspettare in proporzione al lavoro svolto, sintomo di un kernel *Memory-Bound*.

5.4.1 Analisi sul tempo di esecuzione

Come anticipato prima, nonostante i miglioramenti ottenuti, non osserviamo un tempo di esecuzione significativamente minore. Questo fenomeno può essere attribuito a due fattori:

1. **Barriera di sincronizzazione** (`__syncthreads()`): Poiché tutti i thread di uno stesso blocco condividono la shared memory, è necessario assicurarsi che tutti e quanti i thread abbiano caricato i propri dati prima di poter effettuare i calcoli. Questo è fondamentale per evitare *race-conditions* sui dati in memoria condivisa, però introduce latenza perchè thread pronti devono aspettare che tutti gli altri raggiungano la barriera.

2. **Natura del problema:** L'utilizzo della shared memory sicuramente riduce i lunghi accessi in memoria globale. Tuttavia, i calcoli che devono essere effettuati da ogni thread sono talmente semplici che impiegano pochi cicli per essere eseguiti. Il vantaggio di avere dati vicini non è tanto determinante per le operazioni del kernel con questa configurazione di griglia di gioco e generazioni. Inoltre, la Cache L2 è abbastanza veloce, anche se non quanto la Shared Memory. Il suo uso efficiente nelle versioni precedenti (75% circa di hit rate) abbattava i costi di accesso alla DRAM.

5.4.2 Confronto tra versioni 5.4 e 5.3

GRIGLIA DI GIOCO	VERSIONE	T. ESEC. TOTALE	T. GENERAZIONI	SPEED-UP TOT.	SPEED-UP GEN.
1024 – 1000 GEN	Cuda 16x16	306,53 ms	119,308 ms		
	Cuda SMEM	306,12 ms	115,824 ms	1,001x	1,03x
4096 – 1000 GEN	Cuda 16x16	1,254 s	1,078 s		
	Cuda SMEM	1,176 s	0,983 s	1,06x	1,10x
32.768 – 1000 GEN	Cuda 16x16	71,657 s	71,186 s		
	Cuda SMEM	61,026 s	60,547 s	1,17x	1,18x

Come è possibile osservare dalla tabella, più grandi sono le dimensioni della griglia di gioco e il numero di generazioni, più il divario tra l'uso o meno della *shared memory* è evidente.

5.4.3 Bank Conflict in shared memory

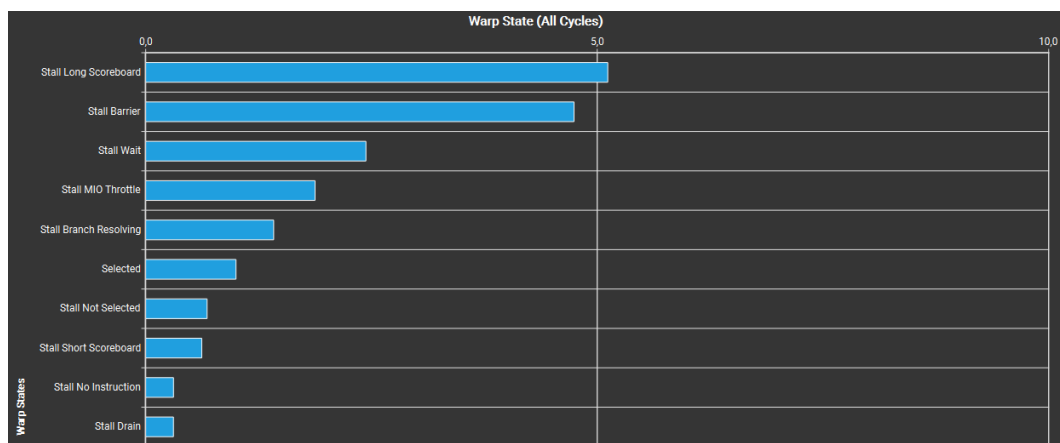
L'utilizzo della Shared Memory comporta un corretto accesso ai 32 bank da parte dei thread dello stesso warp. Considerando la GPU in esame, abbiamo a disposizione 32 bank da 4 byte l'uno. Per un singolo warp, quindi, necessitiamo di 32 byte, cioè 8 bank. Dal momento che ogni thread è interessato ad un solo byte dei 4 presenti in un singolo bank allo stesso indice di riga, i bank conflict risultano essere 0 grazie al *broadcasting*. L'hardware preleverà l'intera *word* dal bank con una sola istruzione e la smisterà in blocchi di byte ai 4 singoli thread, ottimizzando anche la banda a disposizione.

	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	294.912	294.912			
Shared Load Matrix	0	0	294.912	11,66	0
Shared Store	122.880	122.880	122.880	4,86	0
Shared Atomic	0	0	0	0	0
Other	-	-	174.976	6,92	0
Total	417.792	417.792	592.768	23,45	0

5.4.3 Conclusioni

Possiamo concludere che la versione attuale risulta essere la più efficiente nell'utilizzo delle risorse: meno registri richiesti, meno istruzioni, utilizzo della shared-memory e accessi alla memoria più efficienti.

È ragionevole affermare come il kernel sia *Memory-Bound* piuttosto che *Compute-Bound*: Il rapporto tra il tempo di esecuzione delle istruzioni di calcolo e i tempi di attesa dei dati è nettamente sbilanciato verso questi ultimi.



Il grafico sopra conferma questa intuizione: il numero di cicli medio di attesa dovuti a *Long Scoreboard* e alla barriera di sincronizzazione (`__syncthreads()`) sono la principale causa e colli di bottiglia che rendono in stallo i warp.

6. CUDA – Analisi delle prestazioni su diverse griglie e generazioni

In questa parte siamo andati ad eseguire ulteriori test su simulazioni con griglie più grandi e con un numero elevato di generazioni tra i tre paradigmi di implementazione. Come riferimento abbiamo utilizzato: sequenziale presentata nel Cap. 2; CPU SIMD sviluppata nel Cap.3; CUDA kernel basato su Shared Memory ottimizzato nel Cap. 5.4.

GRIGLIA GIOCO	METODO	T. ESECUZIONE TOTALE	T. ESECUZIONE GENERAZIONI	SPEED-UP GENERAZIONI	SPEED-UP TOTALE
1024X1024 100 GEN.	Sequenziale (-O0)	1.253,69 ms	1.228.64 ms		
	SIMD (-O0)	340,17 ms	315.29 ms	3.90x	3.69x
	CUDA	96,78 ms	11.85 ms	103.68x	12.95x
2048X 2048 100 GEN.	Sequenziale (-O0)	5.058,36 ms	4.961.28 ms		
	SIMD (-O0)	1.390,62 ms	1.291.98 ms	3.84x	3.64x
	CUDA	140,22 ms	45.68 ms	108.62x	36.07x
32.768X32.768 100 GEN.	Sequenziale (-O0)	1.411 s	1.385 s		
	SIMD (-O0)	374,6 s	348,2 s	3.98x	3.77x
	CUDA	6,405 s	6,057 s	228.65x	220.25x

Confrontando i risultati presentati nella tabella si nota come l'efficacia delle diverse strategie di approccio al problema cambi in relazione alla dimensione della griglia e numero di generazioni simulate.

In una prima analisi possiamo vedere quanto sia netta l'efficacia del parallelismo CUDA. Essa offre, già a partire da griglie di gioco piccole, speed-up nel calcolo delle generazioni enormemente superiori rispetto al caso SIMD (-O0).

In seconda analisi, osserviamo come l'overhead non parallelizzabile tra gli speed-up nelle varie simulazioni CUDA abbia un impatto variabile. Per griglie di piccola dimensione, infatti, i tempi non legati all'esecuzione del kernel rappresentano una porzione significativa del tempo totale di esecuzione. Questo dimostra come il parallelismo massivo offerto dalle GPU sia sfruttato al meglio quando la *dimensione dati* >> *overhead sequenziale*, il quale diventa trascurabile rispetto al tempo di calcolo totale.

7. Implementazioni grafiche

Abbiamo successivamente realizzato due implementazioni di grafica per la visualizzazione della griglia di gioco del GoL che usano la pipeline CUDA/OpenGL. La differenza chiave tra le due risiede nel come e dove viene eseguita la logica di calcolo delle generazioni e nel conseguente flusso di dati.

7.1 Logica kernel GPU + rendering GPU

In questa configurazione sia la logica del GoL che il rendering vengono eseguiti interamente su GPU, in modo tale da ridurre al minimo la lenta comunicazione Host – Device. Flusso di lavoro:

1. il kernel `gol_step_2d2d` calcola il nuovo stato della griglia di gioco salvandola direttamente in VRAM.
2. Tramite `cudaGraphicsMapResources()`, CUDA “prende in prestito” il buffer grafico *Pixel Buffer Object (PBO)* da OpenGL.
3. Un ulteriore kernel `renderKernel` (stesse dimensioni di griglia e blocchi di `gol_step_2d2d`) legge lo stato della nuova generazione dalla VRAM e li trasforma in colori che inserisce nel PBO.
4. CUDA poi rilascia il PBO che viene ora utilizzato da OpenGL per renderizzare la griglia di gioco a video.

7.2 Logica SIMD CPU + rendering GPU

In questo caso la logica di update viene eseguita sulla CPU tramite funzione `update_with_sse()`. Successivamente, la griglia di gioco aggiornata è copiata dalla memoria host CPU in memoria device GPU con `cudaMemcpy()` (potenziale bottleneck). Il resto del flusso è, infine, uguale allo scenario precedente.

7.3 Analisi comparativa delle performance

GRIGLIA - 1000 GEN.	CPU SIMD + GPU RENDER	FULL GPU	FATTORE LIMITANTE
1024X1024	~ 16,8 secondi	~ 16,8 secondi	Rendering/V-Sync
2048X2048	~ 16,7 secondi	~ 16,7 secondi	Rendering/V-Sync

Possiamo vedere come per griglie relativamente piccole le performance totali siano praticamente identiche. Questo è dovuto al predominio dell’overhead della gestione della finestra e sincronizzazione con il refresh rate del monitor (tempi concordi a 60 FPS) sul calcolo della logica del GoL.

GRIGLIA – 1000 GEN.	CPU SIMD + GPU RENDER	FULL GPU	VANTAGGIO FULL GPU
8192X8192	~ 192 secondi	~ 78 secondi	2.46x Più Veloce

All’aumentare della dimensione della griglia, il bottleneck passa dall’overhead di rendering a quello per il trasferimento dei dati e del calcolo della nuova griglia. In questo scenario, il trasferimento di dati tra Host e Device alla fine di ogni generazione penalizza la versione CPU-SIMD.

